

Project Connect 4 Report

Cheniour Maxime
Cinquanta Octave

February 2020

Contents

1 Introduction	1
2 Contribution	1
3 Analyze	2
4 Students' Feedback	3
4.1 Maxime's Feedback	3
4.2 Octave's Feedback	3

1 Introduction

We were tasked with a project consisting of recreating the classic game 'Connect 4' using the python language. The game was to be displayed inside the terminal. Our assignment was given in two parts : the first one consisting of creating the game grid, being able to display it, enabling players to insert discs into the grid correctly, a way to check whether or not someone has won and announce it and finally a turn system alternating between the two players for the game to take place. The second part was a special addition to the base game, a rotation system meant to complexify it.

2 Contribution

In this section we will now detail how we created the program, and how we coded each and every functionality our game possesses.

Firstly, we prompt the users to enter the length of the grid they desire to play on, which is only one integer since the grid we use is a square. A while loop checks whether the given integer is superior or equal to 4, as playing on a smaller grid would be completely impossible. We then use a function that creates the grid, which is defined using for loops as a two-dimensional array of, for now, -1s. The next function is the one used for displaying the game grid. It uses a

for loop to create strings composed of the different discs. Another for loop is here to create the full line each time, displaying 'X' for 1s, 'O' for 2s and '●' for anything else (in this case its -1s), and the discs are separated by spaces for a better view of the grid.

Secondly, we used a function to define a turn and execute it until the game is won. This function starts by telling whose turn it is, and takes a given integer as the player's move. This integer is reduced by one so that player who do not know that in computer science the first column of an array would be asked using a zero can still play without being confused. The function checks whether the given integer is within the boundaries of the array, aka if the player plays in an existing column. It then checks if the column is full or not using the presence of -1s so that it is possible for the player to add a disc to that particular column. Then the function checks whose turn it was in order to return a turn for the other player. Before returning it displays the grid, asks the player if he wants to use rotation (this will be discussed bellow) and checks the grid to see if a player has won. The function that checks if a winning situation has been reached is composed of four sets of two loops and an if, each checking for a specific type of win. In this game it is possible to win horizontally, vertically, with an ascending diagonal and a descending diagonal. Each if statement checks every group of four values in an order corresponding to the type of win and if those four values are equal it declares a win. The different ranges of the loops are cautiously arranged so that the index never goes out of the array's range.

Finally, we have a function that allows for each player to rotate the grid each round. The function creates a second grid identical to the first one for a later use. It asks the player how many rotations he wants, and it is possible to ask for zero rotation, therefore leaving the grid as it is. If the player chooses to rotate it however, then a loop function using the number of rotations is started. This loop is composed of two systems : one used to create the new and the other to apply gravity to it. The first uses two loop statements to replace the values of the first grid into the second one in respect to a 90° rotation. The second one uses three loops to check whether or not there is space (meaning -1s) bellow each element in order to lower them if necessary. The function then displays the new grid and returns it.

3 Analyze

Our program is probably far from optimized. We reached a total of 105 lines of code, though we could have found ways to minimize the number of lines of our program and thus fasten its execution. However this problem is sort of subverted by the fact that the program's speed difference once optimized would be unnoticed and most likely insignificant. As a matter of optimization, the winner function is four times longer than it could be, though this might be a flaw but also another feature of the program. Distinguishing the four types of win allows for us to input specific win messages as we in fact did. This could be further refined as during the development of the program we had the idea

of inserting an achievement system in the game. This might be irrelevant to a game of this size and scale but it could be nice to add replay-ability by asking the players to try and obtain the four different types of win while playing against each other. This would require use of a global variable to keep track of the wins and of a retry system we could implement asking the players whether or not they wish to stop playing or keep at it. Also, as it is and was asked, the rotation system only features a 90 °right rotation option. It could be a nice improvement to add a feature of left rotations, or a feature to do a 180 °feature (without the gravity in between, therefore reversing the board completely). These additions would complexify the game even more and make for tougher strategic battles to take place. Well, its still Connect 4 and not a game of chess, but upgrades to the game can add a sense of strategy more developed than it is in the base game.

4 Students' Feedback

4.1 Maxime's Feedback

In the end, I really enjoyed this project. It allowed us to utilize our newly learned knowledge from the programming classes and reuse them in our own way. At first, I was a bit lost on what to do and how to do it but in the end I believe that the difficulty was really well gauged, even if a few functions made me pull my hairs out. But this in turn showed me how inefficient and crude my skills were, and allowed me to sharpen them for the future projects, which I really hope will be more often as of this semester. The program ended as a fully functional mess of Spaghetti code, but I felt kinda proud of this first work in Cooperation with Octave, and I am impatient to work with him on the next project.

4.2 Octave's Feedback

Overall, I think this project was a very interesting one. The premise of a 'Connect 4' game didn't really make my blood boil at first since its not really a game I enjoy playing most of the time, and it bores me because of its simplicity and because of how bad I am at it. But trying to recreate it using code was a lot more enjoyable than I could have imagined. It was probably because of the amount of satisfaction succeeding in making something work gave. At times it would be challenging and even frustrating to an extent. But all of that only served as a greater satisfaction in the end. The project's difficulty was alright compared to what we learned so far. It wasn't as easy as it could be finished during the first lecture without a sweat, well at least for us, and it wasn't as hard as there were times we were fully lost and didn't know how to progress. I think it was well-balanced, and the time given was also a rather good estimation, it didn't have us rushing through the code or relax and not do anything for several days. I didn't specifically learn many things in terms of python language during

this project, but my understanding of how I need to think and how to code with someone else on the same project file went up by a bit. Overall, I fairly enjoyed this project and cannot wait to know what the next one has in store for us, hopefully a more advanced game to make.